

Tutorial 3

Database Management Systems (MAC-106)

Sanatan Kumar Gupta

Enrollment ID: 24323035

Mathematics and Computing, UG 2Y

March 20, 2026

1.

- Retrieve the loan numbers for all loans exceeding \$10,000 in amount.

$$\Pi_{\text{loan_number}}(\sigma_{\text{amount} > 10000}(\text{loan}))$$

- Identify the IDs of depositors possessing accounts with balances over \$6,000.

$$\Pi_{\text{ID}}(\sigma_{\text{balance} > 6000}(\text{depositor} \bowtie \text{account}))$$

- Fetch the depositor IDs for individuals with account balances above \$6,000 specifically at the “Uptown” branch.

$$\Pi_{\text{ID}}(\sigma_{\text{balance} > 6000 \wedge \text{branch_name} = 'Uptown'}(\text{depositor} \bowtie \text{account}))$$

2.

- **Relation Schema:** This outlines the static structural blueprint of the relation, specifying its attributes and their corresponding data types.
- **Relation:** This represents the dynamic set of actual data instances (or tuples) that strictly adhere to the established schema at any specific moment.

3.

$R = (A, B, C)$ and $S = (D, E, F)$:

- **Projection:**

$$\{ t \mid \exists q \in r (t[A] = q[A]) \}$$

- **Selection:**

$$\{ t \mid t \in r \wedge t[B] = 17 \}$$

- **Cartesian Product:**

$$\{ t \mid \exists p \in r, \exists q \in s (t[A] = p[A] \wedge t[B] = p[B] \wedge t[C] = p[C] \wedge t[D] = q[D] \wedge t[E] = q[E] \wedge t[F] = q[F]) \}$$

- **Complex Expression:**

$$\{ t \mid \exists p \in r, \exists q \in s (p[C] = q[D] \wedge t[A] = p[A] \wedge t[F] = q[F]) \}$$

4.

Given relations r_1 and r_2 operating on schema $R = (A, B, C)$:

- **Projection:**

$$\{ \langle a \rangle \mid \exists b, c (\langle a, b, c \rangle \in r_1) \}$$

- **Selection:**

$$\{ \langle a, b, c \rangle \mid \langle a, b, c \rangle \in r_1 \wedge b = 17 \}$$

- **Union:**

$$\{ \langle a, b, c \rangle \mid \langle a, b, c \rangle \in r_1 \vee \langle a, b, c \rangle \in r_2 \}$$

- **Intersection:**

$$\{ \langle a, b, c \rangle \mid \langle a, b, c \rangle \in r_1 \wedge \langle a, b, c \rangle \in r_2 \}$$

5.

- **a.** Individuals employed by “FBC”:

$$\{ t \mid \exists w \in \text{works} (t[\text{person_name}] = w[\text{person_name}] \wedge w[\text{company_name}] = 'FBC') \}$$

- **b.** Employees at “FBC” earning over \$10,000:

$$\{ t \mid \exists e \in \text{employee}, \exists w \in \text{works} (e[\text{person_name}] = w[\text{person_name}] \wedge w[\text{company_name}] = 'FBC' \wedge w[\text{salary}] > 10000) \}$$

- **c.** Staff residing on the same street and city as their respective managers:

$$\{ t \mid \exists e \in \text{employee}, \exists m \in \text{manages}, \exists e2 \in \text{employee} (e[\text{person_name}] = m[\text{person_name}] \wedge m[\text{manager_name}] = e2[\text{person_name}] \wedge e[\text{street}] = e2[\text{street}] \wedge e[\text{city}] = e2[\text{city}]) \}$$

- **d.** Staff whose salary exceeds all “SBC” employees:

$$\{ t \mid \exists w \in \text{works} (t[\text{person_name}] = w[\text{person_name}] \wedge \forall w2 \in \text{works} (w2[\text{company_name}] = 'SBC' \Rightarrow w[\text{salary}] > w2[\text{salary}])) \}$$

- **e.** Cities where “SBC” is located:

$$\{ t \mid \forall c \in \text{company} (c[\text{company_name}] = 'SBC' \Rightarrow \exists c2 \in \text{company} (c2[\text{company_name}] = t[\text{company_name}])) \}$$

6.

- **a.** Names of personnel at “FBC”:

$$\{ \langle x \rangle \mid \exists y, z (\langle x, y, z \rangle \in \text{works} \wedge y = 'FBC') \}$$

- **b.** Contact details (name, street, city) for “FBC” staff making more than \$10,000:

$$\{ \langle x, s, t \rangle \mid \langle x, s, t \rangle \in \text{employee} \wedge \exists y, m (\langle x, y, m \rangle \in \text{works} \wedge y = 'FBC' \wedge m > 10000) \}$$

- **c.** Personnel sharing a living arrangement (street/city) with their manager:

$$\{ \langle x \rangle \mid \exists s, t, u, v, w (\langle x, s, t \rangle \in \text{employee} \wedge \langle x, u \rangle \in \text{manages} \wedge \langle u, v, w \rangle \in \text{employee} \wedge s = v \wedge t = w) \}$$

- **d.** Individuals out-earning every “SBC” employee:

$$\{ \langle x \rangle \mid \exists y, z (\langle x, y, z \rangle \in \text{works} \wedge \forall p, q, r (\langle p, q, r \rangle \in \text{works} \wedge q = 'SBC' \Rightarrow z > r)) \}$$

- **e.** Businesses present in all “SBC” municipalities:

$$\{ \langle a \rangle \mid \forall b, c (\langle b, c \rangle \in \text{company} \wedge b = 'SBC' \Rightarrow \langle a, c \rangle \in \text{company}) \}$$

7.

Assuming schemas $r(A, B)$ and $s(A, C)$:

- **a. Projection and Selection equivalence:**

$$\{ \langle a \rangle \mid \exists b (\langle a, b \rangle \in r \wedge b = 17) \} \equiv \Pi_A(\sigma_{B=17}(r))$$

- **b. Join Operation equivalence:**

$$\{ \langle a, b, c \rangle \mid \langle a, b \rangle \in r \wedge \langle a, c \rangle \in s \} \equiv r \bowtie s$$

- **c. Complex Set Operation equivalence:**

$$\{ \langle a \rangle \mid \exists b (\langle a, b \rangle \in r) \vee \forall c (\exists d (\langle d, c \rangle \in s) \Rightarrow \langle a, c \rangle \in s) \} \equiv \Pi_A(r) \cup \left(\Pi_A(r) - \Pi_A((\Pi_A(r) \times \Pi_C(s)) - s) \right)$$

- **d. Complex Join and Selection equivalence:**

$$\{ \langle a \rangle \mid \exists c (\langle a, c \rangle \in s \wedge \exists b_1, b_2 (\langle a, b_1 \rangle \in r \wedge \langle c, b_2 \rangle \in r \wedge b_1 > b_2)) \} \equiv \Pi_A(\sigma_{b_1 > b_2}((\Pi_A(r \bowtie s) \bowtie r) \times r))$$

- **a. Natural Join ($r \bowtie s$):**

$$\{ t \mid \exists p \in r, \exists q \in s (p[A] = q[A] \wedge t[A] = p[A] \wedge t[B] = p[B] \wedge t[C] = q[C]) \}$$

- **b. Left Outer Join:**

$$\{ t \mid (t \in r \bowtie s) \vee (\exists p \in r (\forall q \in s (p[A] \neq q[A]) \wedge t[A] = p[A] \wedge t[B] = p[B] \wedge t[C] = \text{null})) \}$$

- **c. Full Outer Join:** Combines the results of the Left Outer Join defined above with a Right Outer Join, assigning B as null when no corresponding A exists within r .

9.

$$\{ t \mid t \in r \wedge \forall q \in r (t[A] \geq q[A]) \}$$

10.

1. **Domain Membership:** Every value populated in the resulting tuples must originate from the formula's domain, meaning they are either constants explicitly defined in the formula or values actively existing within the relations.
2. **Existential Quantifiers:** Whenever evaluating a subformula $\exists x(P(x))$, if the condition $P(x)$ resolves to true, x must concurrently exist within the active domain.
3. **Universal Quantifiers:** Whenever evaluating a subformula $\forall x(P(x))$, if x falls outside the active domain, the condition $P(x)$ must fundamentally evaluate to true.